

1st_tutorial

April 15, 2026

```
[63]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
[64]: medical_df = pd.read_csv("sklearn_freeCodeAcademy/full_course_Aakash_N_S/
↳insurance.csv", encoding="utf-8-sig", low_memory=False)
```

```
[65]: medical_df
```

```
[65]:
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

```
[66]: medical_df.info()
```

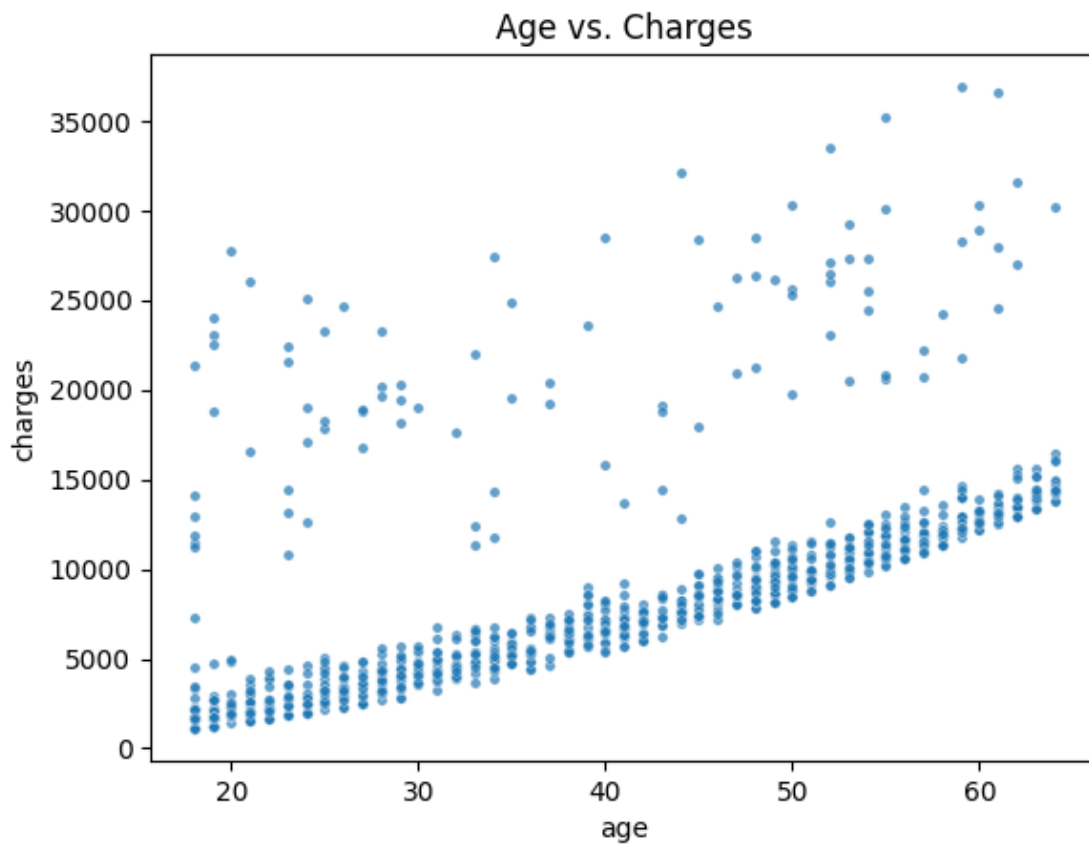
```
<class 'pandas.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   str
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   str
5   region      1338 non-null   str
```

```
6   charges    1338 non-null   float64
dtypes: float64(2), int64(2), str(3)
memory usage: 73.3 KB
```

```
[67]: non_smoker_df = medical_df[medical_df.smoker == "no"]
```

```
[68]: plt.title("Age vs. Charges")
sns.scatterplot(data=non_smoker_df, x="age", y="charges", alpha = 0.7, s=15)
```

```
[68]: <Axes: title={'center': 'Age vs. Charges'}, xlabel='age', ylabel='charges'>
```



```
[77]: def estimate_charges(age,w,b):
      return w * age + b
```

```
[70]: w = 50
      b=100
```

```
[71]: estimate_charges(30,w,b)
```

```
[71]: 1600
```

```
[72]: ages = non_smoker_df.age
ages
```

```
[72]: 1      18
      2      28
      3      33
      4      32
      5      31
      ..
     1332     52
     1333     50
     1334     18
     1335     18
     1336     21
      Name: age, Length: 1064, dtype: int64
```

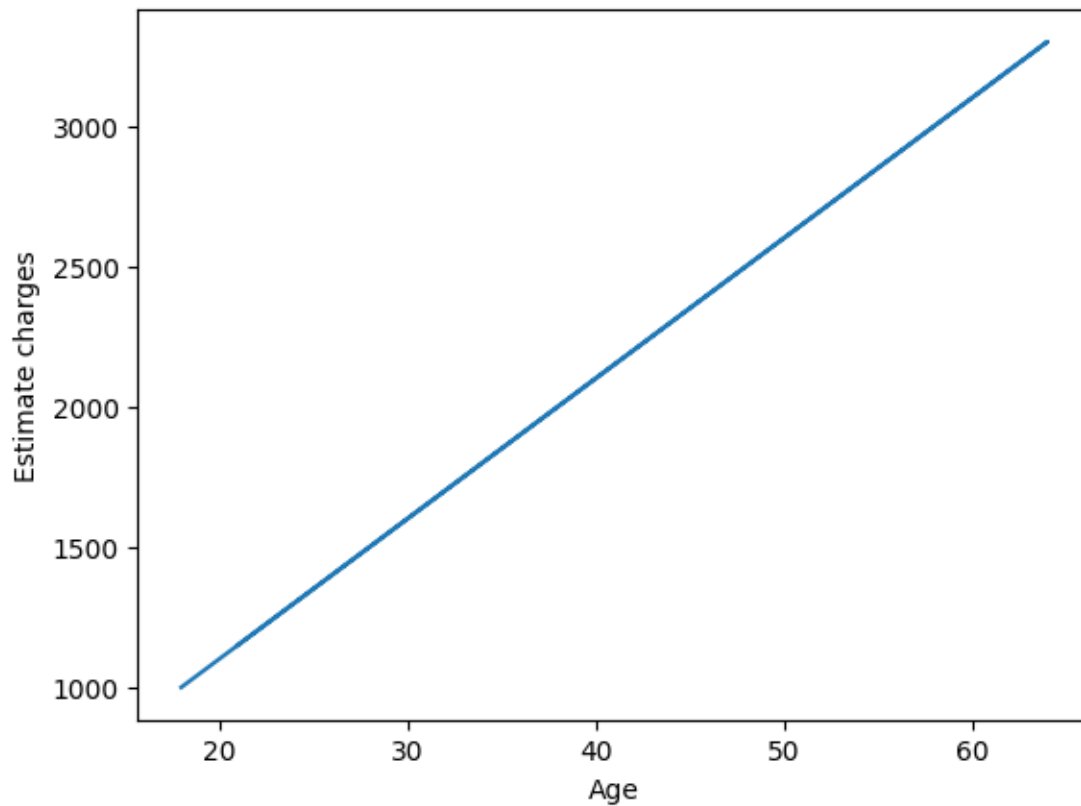
```
[73]: estimate_charges = estimate_charges(ages, w,b)
estimate_charges
```

```
[73]: 1      1000
      2      1500
      3      1750
      4      1700
      5      1650
      ...
     1332     2700
     1333     2600
     1334     1000
     1335     1000
     1336     1150
      Name: age, Length: 1064, dtype: int64
```

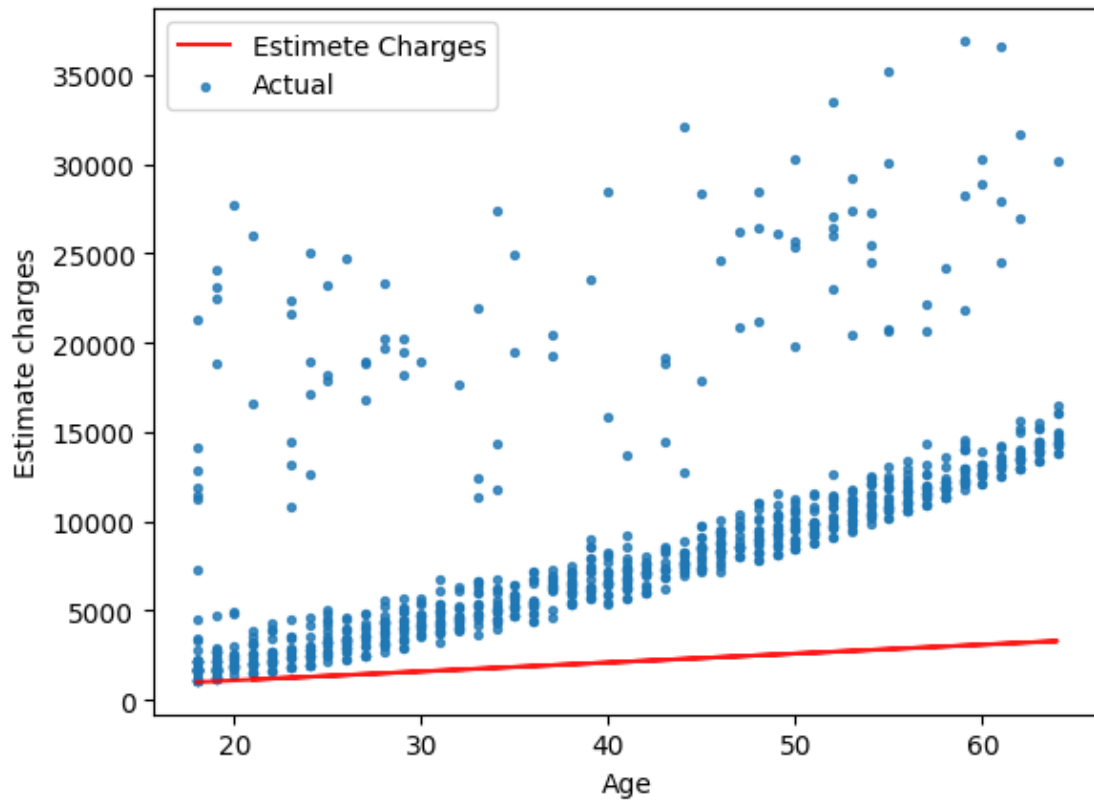
```
[74]: non_smoker_df.charges
```

```
[74]: 1      1725.55230
      2      4449.46200
      3      21984.47061
      4      3866.85520
      5      3756.62160
      ...
     1332     11411.68500
     1333     10600.54830
     1334      2205.98080
     1335      1629.83350
     1336      2007.94500
      Name: charges, Length: 1064, dtype: float64
```

```
[59]: plt.plot(ages, estimate_charges);  
plt.xlabel("Age");  
plt.ylabel("Estimate charges");
```



```
[60]: target = non_smoker_df.charges  
  
plt.plot(ages, estimate_charges, "r", alpha=0.9);  
  
plt.scatter(ages,target, s=8, alpha=0.8);  
plt.xlabel("Age");  
plt.ylabel("Estimate charges");  
plt.legend(["Estimate Charges", "Actual"]);
```

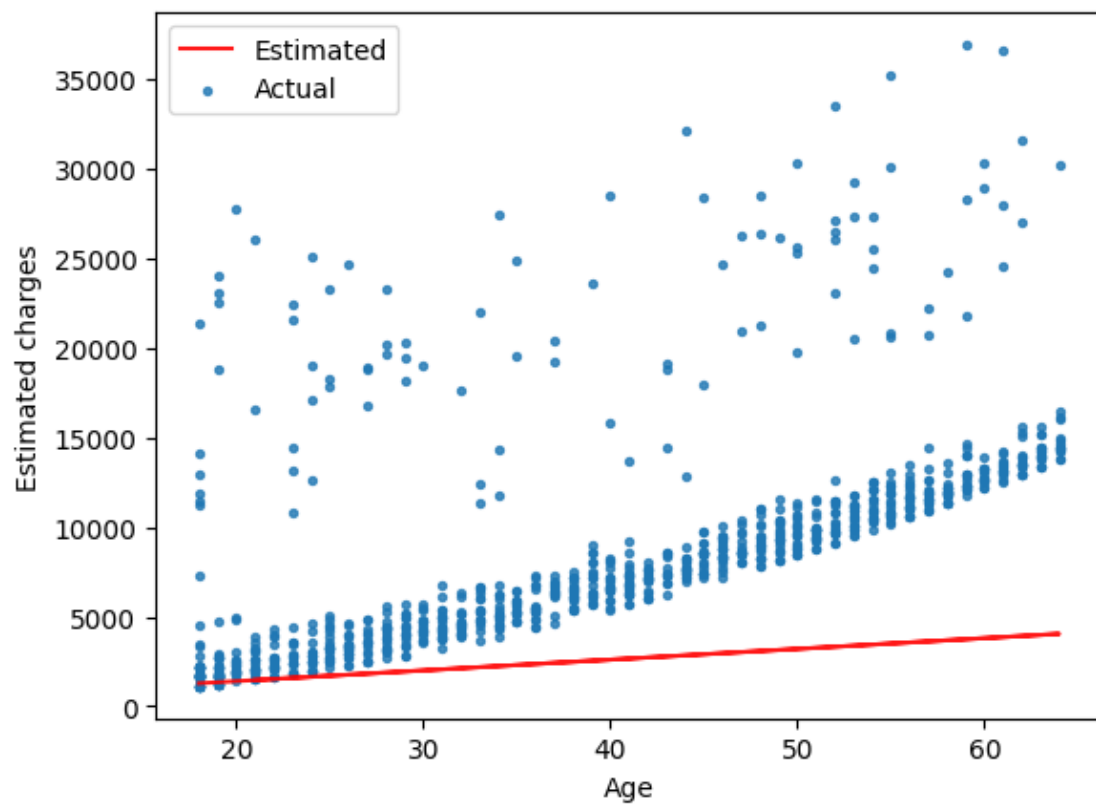


```
[82]: def try_params(w, b):
    ages = non_smoker_df.age
    target = non_smoker_df.charges

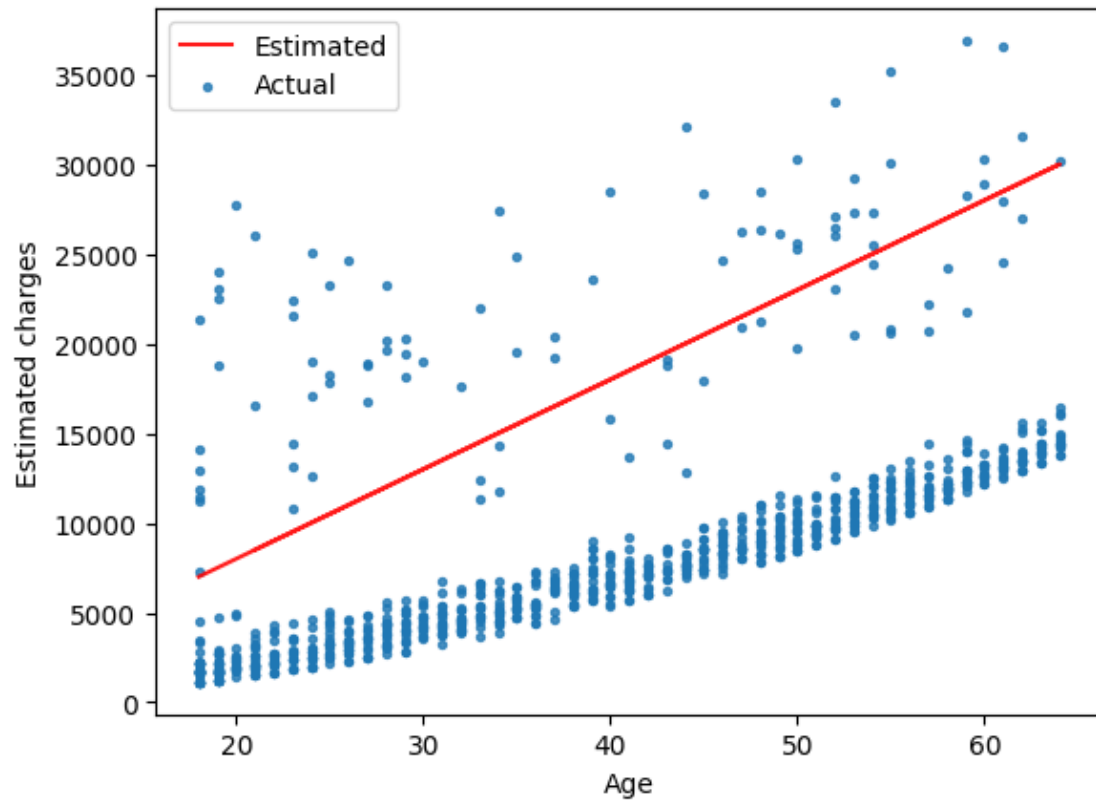
    estimated_charges = estimate_charges(ages, w, b)

    plt.plot(ages, estimated_charges, "r", alpha=0.9)
    plt.scatter(ages, target, s=8, alpha=0.8)
    plt.xlabel("Age")
    plt.ylabel("Estimated charges")
    plt.legend(["Estimated", "Actual"])
```

```
[83]: try_params(60, 200)
```



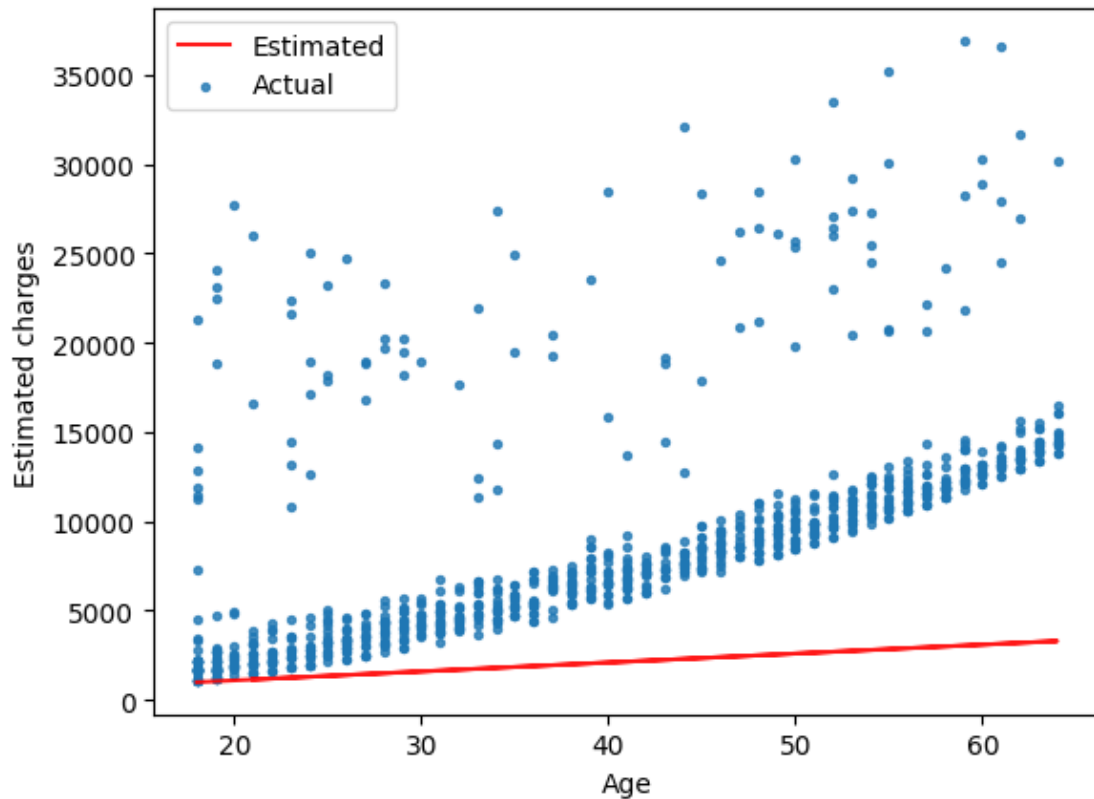
```
[84]: try_params(500, -2000)
```



```
[85]: def rmse(targets, predictions):  
      return np.sqrt(np.mean(np.square(targets - predictions)))
```

```
[86]: w = 50  
      b = 100
```

```
[87]: try_params(w,b)
```



```
[88]: targets = non_smoker_df["charges"]
      predicted = estimate_charges(non_smoker_df.age, w, b)
```

```
[89]: rmse(targets, predicted)
```

```
[89]: np.float64(8461.949562575493)
```

```
[90]: #full block, manually playing with parms
def try_params(w, b):
    ages = non_smoker_df.age
    target = non_smoker_df.charges
    predictions = estimate_charges(ages, w, b)

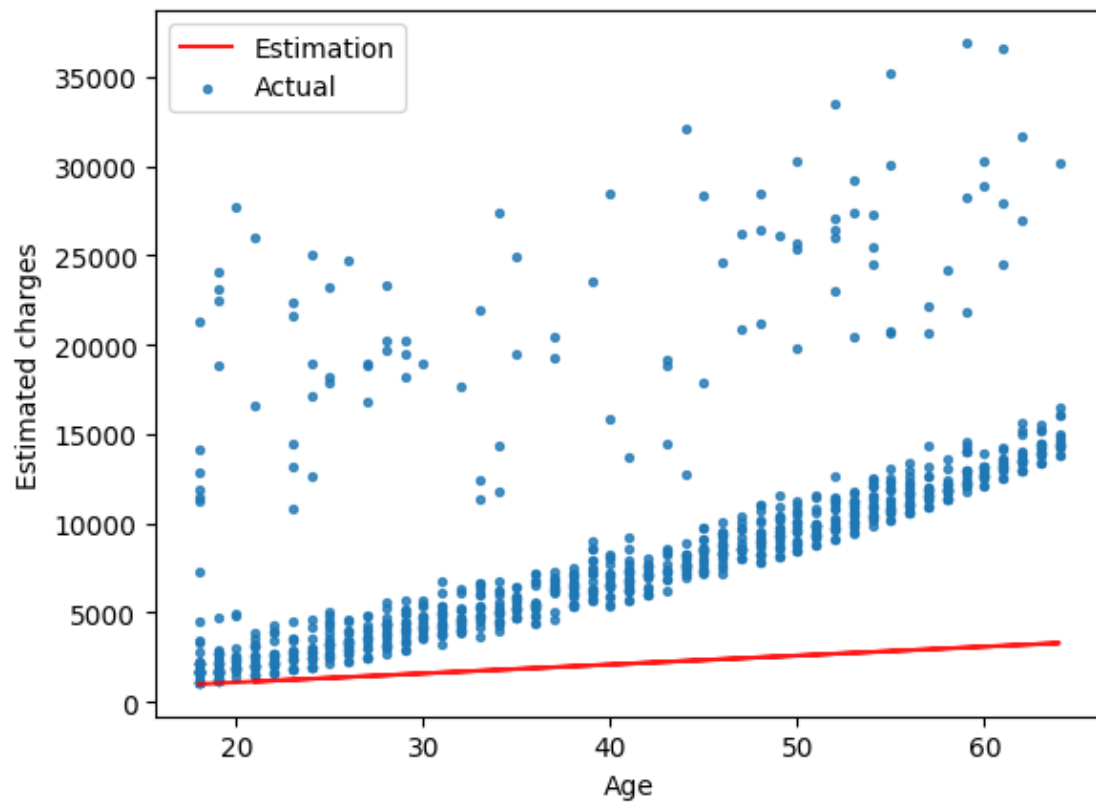
    plt.plot(ages, predictions, "r", alpha=0.9)
    plt.scatter(ages, target, s=8, alpha=0.8)
    plt.xlabel("Age")
    plt.ylabel("Estimated charges")
    plt.legend(["Estimation", "Actual"])

    loss = rmse(target, predictions)
    print("RMSE loss: ", loss)
```



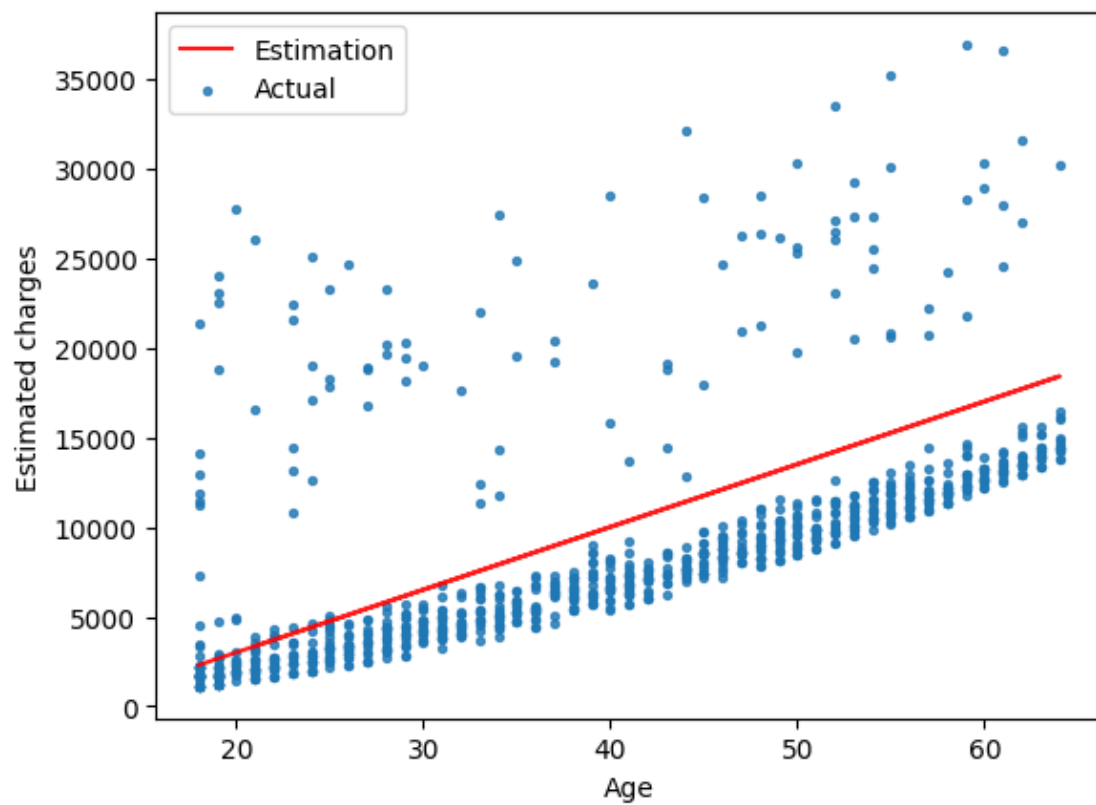
```
[91]: try_params(50,100)
```

RMSE loss: 8461.949562575493



```
[92]: try_params(350, -4000)
```

RMSE loss: 4991.993804156943



[]: